Determining the Unreachable: Constraint-Guided Reachability Analysis for Dependency Vulnerabilities

WENBU FENG, Tianjin University, China
XIAOHONG LI, Tianjin University, China
RUITAO FENG*, Southern Cross University, Australia
YAO ZHANG*, Tianjin University, China
YUEKANG LI, UNSW Sydney, Australia
ZHIPING ZHOU, Tianjin University, China
YUNQIAN WANG, Tianjin University, China
YUQING LI, Tianjin University, China

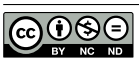
In software development, investigating the accessibility of dependency vulnerabilities is of great importance, as third-party libraries often contain known vulnerabilities that could be exploited in the application's business logic. The existing accessibility analysis methods encounter challenges such as undecidability, abstraction loss, and path explosion in large-scale programs, resulting in an inaccurate distinction between accessibility vulnerabilities and non-accessibility vulnerabilities. This paper introduces an approach called *ConVReach* for analyzing the reachability of vulnerabilities in dependencies in C/C++ programs. *ConVReach* overcomes the problems of high abstraction loss and potential path explosion in the current methods by combining static and dynamic approaches, particularly a constraint-guided analysis method. This approach extracts and decomposes the path constraints that trigger vulnerabilities, independently verifies the satisfiability of each constraint, and then aggregates the feasible paths. This effectively reduces unnecessary path exploration and avoids the common path explosion issues in traditional methods. Experimental results show that *ConVReach* outperforms existing tools in both accuracy and efficiency, effectively distinguishing between reachable and unreachable vulnerabilities, and significantly reducing false positives and false negatives.

We constructed a benchmark dataset to evaluate *ConVReach*, which includes 53 CVEs and 347 flags artificially inserted into various open-source projects. This dataset was designed to simulate both real-world vulnerabilities and complex scenarios. Through testing on this dataset, *ConVReach* demonstrated exceptional performance. It successfully identified 59 out of 61 reachable vulnerabilities and all 23 unreachable ones in the CVE dataset. Within a 24-hour time budget, *ConVReach* detected above 50% more reachable vulnerabilities than the baseline tools in the first 6 hours and nearly completed the detection of reachable vulnerabilities by the 12-hour mark. These results highlight *ConVReach*'s superior ability to handle both real-world vulnerabilities and challenging cases.

CCS Concepts: • **Software and its engineering → Software verification and validation**; *Formal software verification*; • **Security and privacy → Software security engineering**.

*Corresponing Author

Authors' Contact Information: Wenbu Feng, Tianjin University, Tianjin, China, ianx@tju.edu.cn; Xiaohong Li, Tianjin University, Tianjin, China, xiaohongli@tju.edu.cn; Ruitao Feng, Southern Cross University, Gold Coast, Australia, ruitao.feng@scu.edu.au; Yao Zhang, Tianjin University, Tianjin, China, zzyy@tju.edu.cn; Yuekang Li, UNSW Sydney, Sydney, Australia, yuekang.li@unsw.edu.au; Zhiping Zhou, Tianjin University, Tianjin, China, zhou_zhiping@tju.edu.cn; Yunqian Wang, Tianjin University, Tianjin, China, wangyq_0617@tju.edu.cn; Yuqing Li, Tianjin University, Tianjin, China, liyuqing0409@tju.edu.cn.



Additional Key Words and Phrases: Reachability analysis, Software security, Dependency analysis, Program analysis

1 Introduction

In software testing and formal verification, analyzing whether certain code paths or program states are reachable is fundamental for understanding program behavior and correctness [Rosu et al. 2013; Stefănescu et al. 2016]. Applied to security, this concept yields vulnerability reachability analysis, which determines whether identified vulnerabilities can actually be triggered in a program's execution [Böhme et al. 2017]. One prominent example is Software Composition Analysis (SCA), which identifies known vulnerabilities in third-party libraries [Ecl 2021; Dep 2025; Snyk 2023] but typically cannot discern if those vulnerabilities are truly exploitable through the application's own logic [Imtiaz et al. 2021; Zhang et al. 2024a]. In practice, many such library flaws are only triggered under specific execution paths or environmental conditions [Liu et al. 2022]. Without incorporating reachability analysis, security tools may produce excessive false positives and low-priority alerts, overloading teams and wasting resources [Jacobs et al. 2021; Johnson et al. 2013]. By distinguishing which vulnerabilities are actually exploitable from those that are benign, reachability analysis enables more accurate risk assessments, timely patching of critical flaws [Alfadel et al. 2023; Pashchenko et al. 2018, 2020], and the safe de-prioritization of non-reachable cases.

Vulnerability reachability analysis methods can be broadly categorized into static and dynamic approaches, depending on whether the actual execution is required. Static methods rely on source code and vulnerability information without compiling or running the program [Fluid Attacks 2025]. They typically use call graphs or program dependence graphs, but struggle to capture complex control flow and runtime semantics [Zhang et al. 2024a; Zhao et al. 2023]. For example, FASTEN [fas 2024] builds cross-library call graphs to determine if vulnerable functions are reachable, while Vulas [Ponta et al. 2018] combines multiple static analysis methods to locate potentially reachable code, but cannot confirm whether it is actually exploitable. Many static approaches assume the availability of Proofs of Concepts (PoCs), however, vulnerability information is often incomplete [Householder et al. 2020], and PoCs may be missing or even malicious [Yadmani et al. 2022]. Dynamic methods, by contrast, execute programs after static preprocessing. Ducousso [Ducousso et al. 2023] introduced adversarial symbolic execution to mitigate the path explosion problem. Another branch, directed gray-box fuzzing [Böhme et al. 2017], attempts to trigger vulnerabilities by input mutation and validate their reachability. However, dynamic methods can only confirm reachability when a vulnerability is triggered; for untriggered vulnerabilities, it remains unclear whether they are truly unreachable.

Generally, the research field of dependency vulnerability reachability analysis has not yet produced a method capable of accurately distinguishing between reachable and unreachable vulnerabilities in complex programs [Zhang et al. 2024a]. Specifically, the existing methods for general vulnerabilities face the following challenges:

C1: Undecidability and abstraction loss in static analysis. Static reachability analysis methods typically rely on structural abstractions such as call graphs or dependence graphs to approximate potential execution paths. These abstractions are intentionally designed to be conservative, ensuring scalability and coverage across large code bases [Fluid Attacks 2025]. As a result, such methods provide a structural over-approximation of reachability, which is well-suited for identifying potentially affected components, but does not aim to determine the semantic feasibility

of executing a path under concrete runtime conditions. Consequently, when applied to vulnerability reachability, static analysis may report paths that are structurally reachable but infeasible in practice, leading to false positives.

C2: Incomplete and costly observability in dynamic analysis. Dynamic methods, such as symbolic execution and directed fuzzing, can confirm whether a vulnerability is reachable by triggering it at run-time. However, these techniques inherently suffer from incomplete and costly observability [Zhang et al. 2024b]. Because they rely on concrete executions under finite inputs and partial environment configurations, their visibility into the program's full state space is fundamentally constrained. This limitation means that vulnerabilities not triggered during observed executions cannot be conclusively classified as unreachable; rather, they may simply have been missed due to insufficient input generation, unmodeled environmental factors, or nondeterministic scheduling effects.

C3: Path exploration in large-scale programs is computationally prohibitive. Modern software often involves extensive dependencies and complex logic, leading reachability analysis methods to encounter exponential growth in possible execution paths. As a consequence, existing methods struggle to provide timely and resource-efficient analysis in such large-scale settings [Ducousso et al. 2023].

In response to these challenges, we propose a constraint-decomposition and aggregation-based approach for dependency-vulnerability reachability analysis. The key idea is to extract the constraint required to trigger a vulnerability, decompose it into smaller units, evaluate their satisfiability independently, and finally aggregate the satisfiable constraints into a reachable path or infer vulnerability unreachability from unsatisfiable constraints. By performing constraint extraction and decomposition (see § 3.3) at the basic-block level, the method captures fine-grained structural information of the programs. Constraint satisfiability analysis (see § 3.4) enables reasoning about unreachability, while constraint aggregation (see § 3.5) avoid endless path exploration in the exploded search space. This approach addresses the main limitations of traditional vulnerability reachability analysis. It can handle complex program behaviors more accurately than static methods (**C1**), infer unreachability where dynamic methods cannot succeed (**C2**), and avoid the costly path exploration that leads to path explosion (**C3**). Furthermore, it applies statistical principles to infer unreachable paths and ensures that reachability analysis completes within a reasonable time frame.

We constructed a real-world vulnerability dataset consisting of 11 programs, 7 dependencies, and 53 CVEs. To enable broader evaluation, we further augmented the dataset by inserting flags, creating Flagbench. We compared our approach against 2 state-of-the-art reachability analysis tools. The experimental results show that ours achieved 96.7% accuracy in 84 independent real-world tests and 94.5% in 347 flag-inserted cases. It consistently outperformed all other tools, exceeding them by approximately 10% in accuracy and by more than 30% in performance under tight time limits. To further evaluate its performance benefits, we conducted overhead tests by quantitatively varying the locations of vulnerabilities. The results indicate that our approach performs particularly well in more complex program structures with a larger number of path constraints.

In summary, we make the following contributions:

- We propose a framework based on constraint decomposition and aggregation, which extracts and splits path constraints at the basic block level, performs independent satisfiability checks, and aggregates satisfiable constraints into reachable paths or infers unreachability.
- We integrate statistical reasoning to provide reliable conclusions about unreachable paths, ensuring that reachability analysis completes within reasonable time budgets while reducing false positives.

- We created a comprehensive real-world + augmented CVE dataset for reachability analysis, including 11 programs, 7 dependencies, 53 CVEs, and 347 flag cases, enabling systematic evaluation in real-world dependency scenarios.
- We evaluate our approach against state-of-the-art tools, achieving the highest accuracy (96.7% on 84 real-world tests and 94.5% on 347 flag cases), outperforming existing methods by up to 10% in 24h and over 30% under tight time budgets.

2 Background & Motivation

2.1 Reachability Analysis

Reachability analysis of vulnerability determines whether vulnerable code in third-party dependencies can be reached and triggered under realistic execution conditions [Foo et al. 2019; Younis et al. 2016, 2014]. Existing methods can be roughly divided into static and dynamic methods according to whether the target program is executed.

Static methods. Static tools such as FASTEN [fas 2024] and Eclipse Steady [Ecl 2021] build cross-library call graphs to identify whether vulnerable functions are structurally reachable from application code. These approaches are highly effective for dependency impact analysis and vulnerability propagation tracking at scale. However, they are not designed to reason about runtime path feasibility or vulnerability trigger conditions. As a result, while a vulnerable function may be reachable in the call graph, whether it can be exercised under valid inputs remains beyond the scope of such structural analyses.

Dynamic methods. Dynamic methods [Böhme et al. 2017; Chen et al. 2018; Huang et al. 2022; Kim et al. 2023; Yu et al. 2024; Zong et al. 2020] compile and execute the code of the main program with its dependencies, exploring whether the vulnerability can be reached by the runtime, with the known vulnerability location as a guiding target. Although these methods can confirm reachability by triggering vulnerabilities, they cannot conclusively prove unreachability due to incomplete input generation and partial environment modelling. Despite these problems, vulnerability reachability analysis is still an important threat analysis task in the software supply chain [Shouling Ji et al. 2022].

2.2 Problem Definition

In this section, we introduce the definitions of our work.

Definition 1 (Vulnerability Reachability Problem)**.** Given a program or component $\mathcal{X}$, States($\mathcal{X}$) denotes the set of all reachable execution states of $\mathcal{X}$. Each state captures the current control location together with the valuation of program variables and relevant memory.

Let $\mathcal{P}$ denote the host program and $\mathcal{L}$ a dependent library as a dependency of $\mathcal{P}$ that contains a known vulnerable state $V \subseteq \text{States}(\mathcal{L})$. We denote by $\text{Paths}(\mathcal{P} \rightsquigarrow \mathcal{L})$ the set of all feasible execution paths that start from an initial state of $\mathcal{P}$ and reach a state in $\mathcal{L}$. Given the input domain $\mathcal{I}$ of $\mathcal{P}$, the dependency vulnerability reachability problem can be defined as:

$$\mathcal{V}\text{Reach} = \langle \mathcal{P}, \mathcal{L}, V, \mathcal{I}, C_P, C_V \rangle \tag{1}$$

where:

- C_P: the *path constraint set*, i.e., the conjunction of control- and data-flow predicates that must hold for a path $\pi \in \text{Paths}(\mathcal{P} \rightsquigarrow \mathcal{L})$ to be feasible;
- C_V: the *state constraint set*, describing the logical predicates that characterize the vulnerability triggering condition within $\mathcal{L}$;
- $\mathcal{I}$: the input domain that determines the feasible valuations of C_P and C_V.

The vulnerability is *reachable* if and only if there exists an input $i \in \mathcal{I}$ and a feasible inter-component path π such that:

$$i \models C_P^{\pi} \ \wedge \ C_V \tag{2}$$

Intuitively, the satisfiability of the conjunction $C_P^{\pi} \wedge C_V$ determines whether the vulnerable state can be reached and triggered through some feasible execution path under valid program inputs.

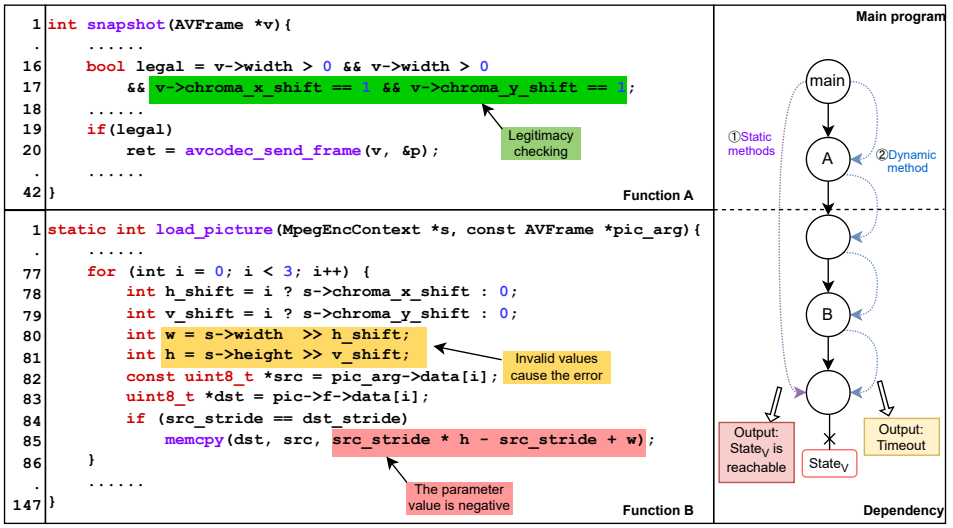


Fig. 1. A motivating example. The left shows the library code (ffmpeg) and the main program's input validation that prevents the vulnerability from being triggered. The right highlights limitations of existing methods: static analysis produces false positives by ignoring trigger conditions, while dynamic analysis can only confirm reachability if the vulnerability is triggered.

2.3 Motivating Example

A dependency vulnerability is not always reachable from the calling program, even when a direct call chain exists. Fig. 1 illustrates a simplified example involving a vulnerability in ffmpeg. In the library code (lower left), setting the parameter `chroma_x_shift` to an invalid value triggers a right-shift exception in function B (`load_picture`), which sets internal variables w and h to zero. This leads to a negative argument in a subsequent call to `memcpy`, resulting in a memory error. However, the main program (upper left) performs a validity check before passing inputs to the library, preventing any data that could trigger the vulnerability from reaching the library. As a result, the vulnerability is unreachable in practice.

2.3.1 Technical Challenges Illustrated by the Example. Conventional **static analysis** tools typically construct a call graph from the host program $\mathcal{P}$ to the vulnerable function in $\mathcal{L}$. While this captures the inter-component call relations (i.e., potential paths $\pi \in \text{Paths}(\mathcal{P} \rightsquigarrow \mathcal{L})$), it cannot reason about the precise satisfiability of the path constraint C_P^{π} or the runtime triggering constraint C_V. In this example, a conditional check in $\mathcal{P}$ prevents an invalid `chroma_x_shift` value, yet static analysis lacks sufficient information to determine whether $\exists i \in \mathcal{I} : i \models C_P^{\pi}$ actually holds. As a

result, it conservatively assumes $\mathrm{SAT}(C_P^\pi)$ = true for all paths, thus overestimating vulnerability reachability and reporting false positives (**C1**).

By contrast, **dynamic analysis** methods, such as concolic execution or directed fuzzing, can confirm reachability only when a vulnerability is concretely triggered, i.e., when $\exists i^* \in \mathcal{I}$ such that $i^* \models (C_P^\pi \wedge C_V)$. In this example, although the execution along function A (`snapshot`) reaches the call site of function B that invokes the vulnerable code, the resulting runtime state no longer satisfies C_V, meaning that $\mathrm{SAT}(C_P^\pi)$ holds but $\mathrm{SAT}(C_V)$ does not. Hence, no conclusive reachability result can be drawn within the given exploration time (**C2**).

Moreover, the large input space $\mathcal{I}$ and complex control-flow structure of $\mathcal{P}$ may lead to an exponential number of feasible paths, making it intractable to enumerate or test all possible combinations. This *path explosion* problem makes it infeasible to determine whether any $i \in \mathcal{I}$ satisfies $(C_P^\pi \wedge C_V)$ in finite time (**C3**). Such incompleteness implies that dynamic methods cannot safely conclude $\neg\mathrm{Reach}(\mathcal{P}, \mathcal{L}, V)$ when a vulnerability is untriggered.

In summary, static methods fail to accurately model the runtime state of $\mathcal{L}$, because they lack evaluation of C_V, while dynamic methods cannot establish unreachability for unexplored inputs, because they lack completeness over $\mathcal{I}$ and Paths. This dichotomy highlights the fundamental challenges of reachability analysis in this example.

2.3.2 Inspirations for Our Work. One central challenge in dependency vulnerability reachability analysis lies in the *compositional complexity of constraints* that must simultaneously hold for a vulnerability to be triggered. A vulnerability is not merely reachable by identifying a feasible call chain $\pi \in \mathrm{Paths}(\mathcal{P} \rightsquigarrow \mathcal{L})$, but only when there exists an input $i \in \mathcal{I}$ such that $i \models (C_P^\pi \wedge C_V)$. In practice, these constraints capture interdependent conditions from multiple dimensions, including input validation, control- and data-flow predicates, and environment configuration. The mutual dependencies among C_P^π and C_V lead to a high-dimensional constraint space that static or dynamic methods alone struggle to analyze effectively.

Static analysis tools tend to over-approximate by relaxing or ignoring complex constraint dependencies, yielding $\mathrm{SAT}(C_P^\pi)$ = true even when no feasible input exists, which causes false positives and imprecise reachability reports. Conversely, dynamic approaches rely on concrete execution to validate $i \models (C_P^\pi \wedge C_V)$, but face severe scalability issues due to the exponential growth of the path space: $|\mathrm{Paths}(\mathcal{P} \rightsquigarrow \mathcal{L})| = O(2^{|B|})$, where $|B|$ denotes the number of branch nodes in $\mathcal{P}$. Such path explosion renders exhaustive exploration infeasible in realistic software systems.

Given these challenges, the key to improving vulnerability reachability analysis is to balance *scalability* and *accuracy* in reasoning about complex constraint systems. The first step is to decompose the large, interdependent constraint set C_P into smaller path-specific subproblems $\{C_P^{\pi_1}, C_P^{\pi_2}, \dots\}$, allowing the analysis to focus independently on feasible segments of program execution while maintaining global soundness. Meanwhile, the computational cost of exploration can be reduced by pruning paths whose constraints are unsatisfiable, i.e., $\exists c \in \Gamma_\pi, \forall i \in \mathcal{I} : \neg(i \models c)$, thereby avoiding exhaustive traversal and prioritizing paths with a higher likelihood of triggering C_V. Through this constraint-guided divide-and-conquer strategy, the analysis not only improves precision by respecting real constraint dependencies, but also achieves scalable reasoning across large, complex software systems.

3 Approach

3.1 Problem Formulation

To overcome the scalability and precision limitations of purely static reachability analysis, we propose *ConVReach*, a *Constraint-Guided Reachability* framework which combines static constraint decomposition with dynamic constraint validation to determine the satisfiability of $C_P^\pi \wedge C_V$. Instead

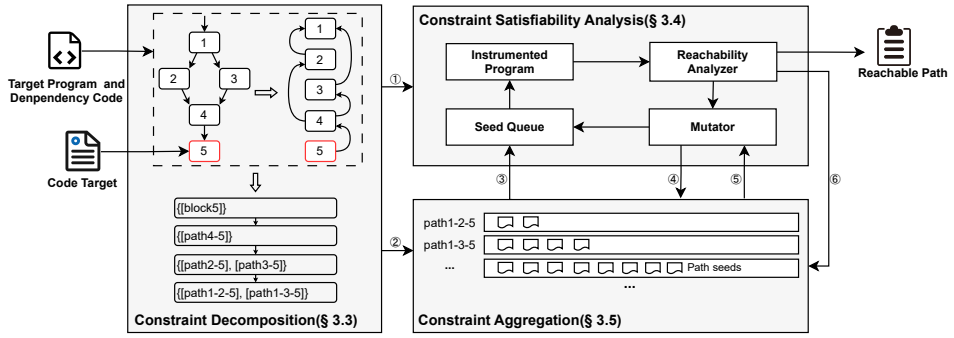


Fig. 2. Overview of *ConVReach*

of exhaustively enumerating all paths, *ConVReach* incrementally explores only constraint-feasible paths that are most likely to lead to vulnerable states.

Given the formal model $\mathcal{V}\text{Reach} = \langle \mathcal{P}, \mathcal{L}, V, \mathcal{I}, C_P, C_V \rangle$, *ConVReach* aims to efficiently decide whether the following satisfiability formula holds:

$$\exists i \in \mathcal{I}, \pi \in \text{Paths}(\mathcal{P} \rightsquigarrow \mathcal{L}) : i \models C_P^{\pi} \wedge C_V \tag{3}$$

To enable scalable reasoning, we decompose C_P and C_V into a set of individual atomic constraints $\{c_1, c_2, \ldots, c_n\}$, where each c_j represents a single constraint. We conduct separate analyses for each c_j:

$$\text{Find } (i_j^*) \text{ s.t. } i_j^* \models c_j,\ i_j^* \in \mathcal{I},\ c_j \in C_P \wedge C_V \tag{4}$$

then we reformulate it as a *Constraint-Guided Reachability* problem:

$$\text{Find } (\pi^*, i^*) \text{ s.t. } i^* \models \bigwedge_{c_j \in C_P^{\pi^*} \wedge C_V} c_j,\ i^* \in \text{Merge}(i_j^*),\ \pi^* \in \text{Paths}(P \rightsquigarrow \mathcal{L}),\ c_j \in C_P \wedge C_V \tag{5}$$

where $Merge()$ represents the cross-fusion of i_j^* in formula 4. This formulation shifts global reachability reasoning into a constraint-driven search space that preserves precision while ensuring scalability.

3.2 Overview

As illustrated in Fig. 2, *ConVReach* takes the source code of a target program and its dependency code with code target as input, and determines if vulnerable code in the dependency can be triggered.

Module 1: Constraint Decomposition (§ 3.3): *ConVReach* firstly uses the Interprocedural Control-Flow Graph(ICFG) to represent and abstract the source code of the target program and its dependencies. Taking the known dependency vulnerability as the code target, all feasible paths between the dependency entry (which refers to the callable APIs of the dependency) and the vulnerability block are analyzed to capture the path constraints and variable constraints related to the vulnerability trigger conditions. These constraints are decomposed into path-specific sets (e.g., $path_{1-2-5}$), enriched with variable and state information, and will serve as the input of satisfiability analysis and aggregation (step ① and ②).

Module 2: Constraint Satisfiability Analysis (§ 3.4): Each path constraint is adopted as a guide for the path seed generation and pruning according to its satisfiability. All seeds (initially placed in the seed queue) are executed on the instrumented program using a mutator to refine seeds to explore nearby paths (i.e., fuzzing). The reachability analyzer determines whether a vulnerability

is reachable according to whether there is a seed that satisfies all the constraints of a path. If satisfiable, the corresponding path seed is recognized to be partially reachable and placed into the path seed pool through step ⑥. For unsatisfiable constraints, the corresponding paths are pruned to reduce the overhead.

Module 3: Constraint Aggregation (§ 3.5): Seeds produced along the same path satisfying different partial constraints are aggregated into new candidate inputs through crossover supported by the mutator (step ④ and ⑤). These aggregated seeds are given higher execution priority and passed back to the seed queue in the Constraint Satisfiability Analysis Module via step ③. This iterative feedback allows the system to gradually converge on high-potential paths, even if they require multiple partial solutions to emerge.

Algorithm 1: Backward Data Flow Analysis

Input: $\mathcal{G}$: Interprocedural Control Flow Graph(ICFG)
$\mathcal{L}_t$: Target vulnerability location
$\mathcal{L}_c$: Current call-layer location
$\mathcal{V}_c$: Initial vulnerability trigger constraint
Output: C_s: Set of constraints in disjunctive normal form

$\mathcal{W} \leftarrow \{(\mathcal{L}_t, \mathcal{V}_c, [])\}$ // Worklist;
$C_s \leftarrow \emptyset$ // Set of constraints for the final output ;
$\mathcal{H} \leftarrow \emptyset$ // The set of hashes, used to determine if visited;
while $\mathcal{W} \neq \emptyset$ **do**
  $(n, \varphi, \pi) \leftarrow \text{pop}(\mathcal{W})$ // Current node, data constraint φ and path constraint π;
  $(\varphi', \pi', U_r) \leftarrow ProcessBlockBackward(n, \varphi, \pi)$;
  // U_r will be True if the next step is a structure that is unable to be resolved by the static part
  if U_r **then**
    $C_s \leftarrow C_s \cup \{(\pi', \varphi')\}$;
    $\mathcal{L}_c \leftarrow \mathcal{L}_c \cup \{n\}$;
    continue;
  end
  if $n \in \mathcal{L}_c$ **then**
    $C_s \leftarrow C_s \cup \{(\pi', \varphi')\}$;
    continue;
  end
  foreach $n' \in Predecessors(\mathcal{G}, n)$ **do**
    $\epsilon \leftarrow GetEdgeCondition(n', n)$;
    $\epsilon' \leftarrow Substitute(\epsilon, \varphi')$;
    $\pi'' \leftarrow [\epsilon'] + \pi'$;
    $h \leftarrow Hash(n', \pi'')$;
    if $h \in \mathcal{H}$ **then**
      continue;
    end
    $\mathcal{H} \leftarrow \mathcal{H} \cup \{h\}$;
    $\mathcal{W} \leftarrow \mathcal{W} \cup \{(n', \varphi', \pi'')\}$;
  end
end
return C_s;

3.3 Constraint Decomposition

The objective of this module is to extract the constraints that characterize the conditions for triggering a vulnerability and group them according to control-flow paths. *ConVReach* achieves

this through the backward data flow analysis over the ICFG (see Algorithm 1). Starting from the vulnerability location and tracing backward toward dependency entry functions, *ConVReach* collects both path constraints (branch predicates) and data flow constraints (variable definition and update). Since variable names differ across functions, all constraints are normalized to the variables visible at dependency entry functions, ensuring consistency for subsequent processing.

3.3.1 Worklist Strategy. As shown in Algorithm 1, the backward data flow analysis is implemented using a worklist $\mathcal{W}$ and a visited set $\mathcal{H}$ to ensure systematic exploration and avoid redundant processing (Lines 1–3). Initially, the basic block containing the vulnerability is pushed onto $\mathcal{W}$, and $\mathcal{H}$ is empty. While $\mathcal{W}$ is not empty, the algorithm pops a block and processes it (Lines 4–5). If the block has already been visited, it is skipped to prevent infinite loops (Lines 21–24). This strategy guarantees that all feasible predecessor paths are explored in a depth-first or breadth-first manner, depending on the implementation. For each current block n and its path constraints π and data constraints φ, the algorithm calls the *ProcessBlockBackward* function to extract constraints from that block (Line 6). This function checks the instructions of the block for any constraints or data definitions related to the backward slicing.

3.3.2 Handle Nonresolvable Structures. As mentioned in challenges (see § 1), static analysis inherently faces the undecidability barrier. It can never perfectly reconstruct actual runtime states, and the reliability of precise reachability analysis is fundamentally undermined. It is precisely because of these inherent limitations that the algorithm introduces the flag U_r to denote nonresolvable cases. When U_r becomes true (Lines 7–10), the analysis acknowledges that further exploration would exceed what can be soundly or tractably derived from static methods. For example, loops whose iteration counts or termination conditions depend on external events cannot be statically unrolled, and indirect function calls whose targets are determined dynamically (e.g., via function pointers or virtual dispatch) cannot be reliably resolved. In such situations, the algorithm adopts a conservative extraction strategy: it halts backward exploration at the unresolved point, records the constraints gathered so far into the global set C_s, and marks the location as a call-layer or dependency entry point. This treatment makes the analysis boundary explicit: all constraints up to the frontier are preserved, while conditions beyond it are recognized as undecidable under static abstraction. By doing so, the analysis neither speculates nor over-approximates, but instead clearly delineates the tractable scope of path and data flow constraint extraction.

3.3.3 Backward Traversal and Constraint Propagation. After extracting intra-block constraints, the analysis traverses antecedent blocks in the ICFG, moving step by step backward along the path (Lines 18–24). At each step, it incorporates both path constraints from branch conditions and data flow constraints from variable definitions or updates, thereby propagating constraints upstream. For instance, in Fig. 3, the analysis begins at block 5 with the assertion Assert, $r < 32$. Negating it yields the trigger condition $r \geq 32$, which is added to the constraint set. Traversing to block 4, the branch condition $r\%7 == 0$ introduces a new path constraint, while the assignment $r = r + b$ redefines r and updates the data flow constraints, resulting in $r + b \geq 32$, $(r + b)\%7 == 0$ (step ①). When a block has multiple predecessors, the traversal follows each incoming edge recursively, typically in depth-first order (step ② and ④). Paths are explored one by one, constraints are updated through substitution, and unresolved cases are checked as before. When encountering a resolvable call relation in the ICFG, we replace parameters in the constraints with the corresponding arguments. The analysis then continues into each caller, treating each as a separate path. By the time the worklist is exhausted, all feasible backward paths have been explored and their constraints accumulated. In this way, the algorithm systematically performs antecedents traversal and constraint propagation,

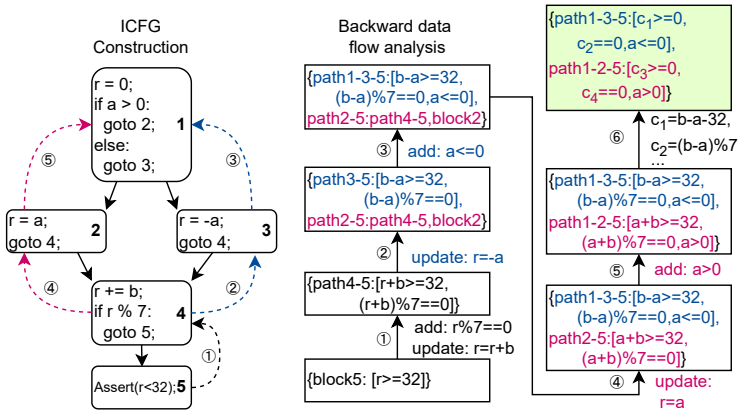


Fig. 3. Constraint decomposition workflow. The left shows the ICFG, where black arrows indicate control flow between basic blocks, and dashed colored arrows (with circled numbers) denote the steps of backward data-flow analysis. The right shows the detailed decomposition process, where constraints are calculated step by step; the colors match the traces on the left, and the circled numbers align with the corresponding analysis steps.

collecting a collection of constraints from the vulnerability site to potential dependency entry points such as user input, function arguments, or nonresolvable structures.

Backward data flow analysis terminates when all paths have been fully explored to their dependency entry points. At this point, the algorithm outputs the constraints collected for each distinct path. Each output is typically a set of constraints (or an equivalent logical formula), and all constraints consisting of multiple variables are converted to single variables to facilitate subsequent evaluation (step ⑥ in Fig. 3). If the vulnerability can be reached through multiple call paths or control flow paths, the analysis yields multiple sets of such constraints, one for each path. In the instrumentation phase before executing the program, we insert the logic for the computation and output of these vulnerability constraint variables into the dependency entry function.

3.4 Constraint Satisfiability Analysis

In order to clearly show the process of constraint satisfiability analysis, we break down this module into three main parts: satisfiability judgment, data collection and management, and unsatisfiability judgment. Satisfiability judgment is the basic process of this part. On this basis, the state data of the program are collected and sorted out, and finally the unsatisfiability judgment of some constraints is carried out, so as to accurately judge the reachability and unreachability. The Fig. 4 illustrates the architecture and execution process of the process. First, the instrumentation program receives the initial seeds as input. The program state is then sent to the satisfiability analysis module, which consists of the basic loop from steps ① to ④. Depending on the different cases of the number of constraints being satisfied, step ② is replaced by either step ⑤ or ⑥, and the unsatisfiable constraint information is passed to the subsequent processing through step ⑦.

3.4.1 Satisfiability Analysis. In the part of constraint satisfiability analysis, we adopt the idea of fuzzing test as a whole. The loop structure is composed of an instrumentation program, a seed queue, a mutator and a reachability analyzer. This loop structure enables the system to evaluate the satisfiability of each constraint by obtaining the values of constraint variables in each test.

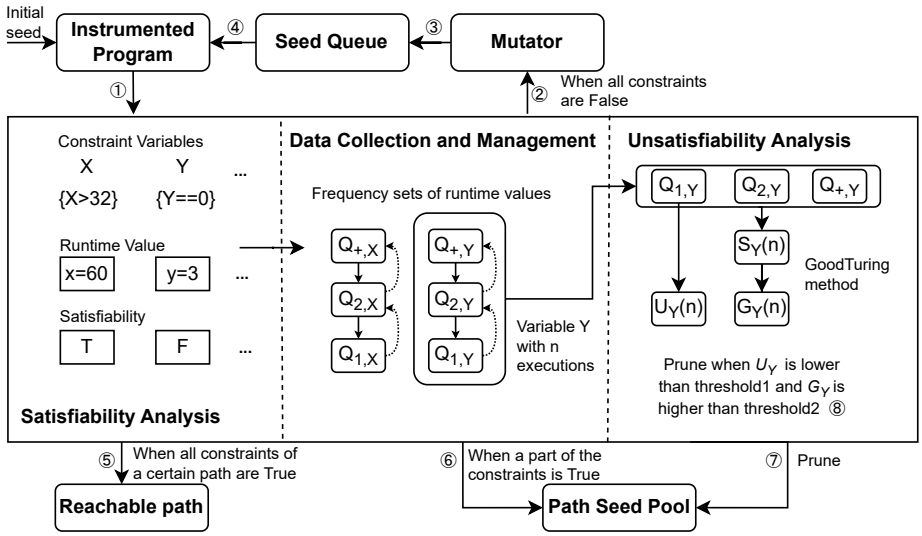


Fig. 4. Constraint satisfiability analysis

As shown on the left side of Fig. 4, the instrumentation program is obtained by instrumenting the detection logic of all constraint variables after extracting constraints by the Constraint Decomposition module from the target program. At each execution, the instrumented program collects the execution state (x, y in Satisfiability Analysis of the Fig. 4) of the path constraint variables (X, Y in Satisfiability Analysis of the Fig. 4) and passes it to the reachability analyzer for processing (step ①). If all constraints are not satisfied, the seed will be sent to the mutator as in step ② for more destructive mutation to explore other potential paths. The mutator will generate new test inputs by regularly mutating the seeds from different sources (step ③). The seed queue stores the seeds to be executed and passes them to the instrumentation program for execution (step ④). The reachability analyzer checks whether each constraint is satisfied (T for True or F for False). As in step ⑤, if all constraints are satisfied, the path is marked as reachable and the vulnerability is judged to be reachable. As in step ⑥, if only part of the constraints are satisfied, the seed is put into the path seed pool of the corresponding path. This procedure ensures that each constraint is fully tested, leading to an efficient satisfiability analysis.

3.4.2 Data Collection and Management. In satisfiability determination, information about constraint variables is collected to determine the satisfiability of constraints. Each time the seed is executed, the satisfaction of the constraints and the execution status are recorded and stored in a continuously updated data structure. The values of constraint variables and their occurrence frequencies are tracked for subsequent unreachability analysis.

When a seed is executed, the instrumentation program records the constraint variable values. For the values of constraint variables that do not satisfy the constraints, the reachability analyzer records their variable values into the continuous frequency tracking structure of each constraint variable. Specifically, for the constraint variable X in Fig. 4, the values x encountered in fuzzing can be classified into the following categories:

- If $x \in Q_{+,X}$, it has been observed three or more times and does not change any set.

- If $x \in Q_{2,X}$is observed three times, it will be moved to $Q_{+,X}$.
- If $x \in Q_{1,X}$is observed twice, it will be moved to $Q_{2,X}$. Otherwise, this is the first time the value has been observed, and x will be added to $Q_{1,X}$.

After obtaining a value y of variable Y, we will check whether it is in the set Q in turn. First, we will check $Q_{+,Y}$, if it is not in $Q_{+,Y}$, we will check $Q_{2,Y}$, and then $Q_{1,Y}$ (the solid arrows in Data Collection and Management of the Fig. 4). After detecting the presence of y in $Q_{2,Y}$ or $Q_{1,Y}$, move y to the next set according to the rule (the virtual arrows in Data Collection and Management of the Fig. 4). This is done for each constraint variable. These frequency sets provide a representation of the value diversity of each variable, and when no variable in a seed satisfies the constraint and no new value is obtained, the seed will not be returned to the loop but discarded.

3.4.3 Unsatisfiability Analysis. After n runs (execution exceeds a certain percentage of the time limit), we use the Good-Turing approximation method [Böhme 2018] to estimate the probability $U_Y(n)$ that the value generated by the constraint variable Y in the next run is a value that has not occurred before. When used in practice, the calculation method of $U_Y(n)$ is:

$$\hat{U}_Y(n) = \frac{q_{1,Y}}{n}. \tag{6}$$

Where $q_{1,Y}$ is the size of $Q_{1,Y}$. When $\hat{U}_Y(n)$ (observed value of $U_Y(n)$) is large, many possible values of Y representing the constraint variable are still unexplored, indicating that the testing progress of Y has not reached the late stage, and the unreachable judgment cannot be given temporarily. In contrast, when the value of $\hat{U}_Y(n)$ decreases below a certain threshold, we can consider that the exploration of Y becomes more difficult to produce new values. If we focus only on $U_Y(n)$, we might stop too early once its value drops, even if we only cover 50% of the total available values. To measure the overall coverage, we apply the Chao2 lower bound estimator to predict the total number of S values:

$$S_Y(n) = q_{1,Y} + q_{2,Y} + q_{+,Y}. \tag{7}$$

$$\hat{S}_Y(n) = S_Y(n) + \frac{q_{1,Y}^2}{2q_{2,Y}}. \tag{8}$$

The coverage ratio:

$$G_Y(n) = \frac{S_Y(n)}{\hat{S}_Y(n)} \tag{9}$$

Measure the proportion of the value of Y in the exploration space. $G_Y(n)$ tells us how much of the total estimate space has been covered [Böhme 2018; Böhme et al. 2021]. Even if $\hat{U}_Y(n)$ is small or even zero, $G_Y(n)$ can still be small if the space is large; in contrast, $G_Y(n)$ can approach 1.0 even if $\hat{U}_Y(n)$ remains non-zero, indicating that the space is nearly saturated. As shown by ⑧ in Fig. 4, only if $G_Y(n)$ is greater than a threshold (we have covered most of the space) and $\hat{U}_Y(n)$ is less than another threshold can we conclude that this constraint is most likely unsatisfiable. Based on the original paper of this method [Böhme 2018] and a series of tests on the dataset of § 5.2 by changing the threshold parameters, we finally determined the threshold value of U as 0.01 and the threshold value of G as 0.95, and this threshold will be relaxed to 0.05 and 0.9 as the remaining time budget is reduced to avoid spending more time on this constraint. Once any constraint in a target path is marked as unsatisfiable in this way, the path is considered unreachable, the reachability analyzer immediately prunes the entire path from further tests, and all constraint variables related only to that path will no longer continue to be computed and recorded. When all paths of a vulnerability

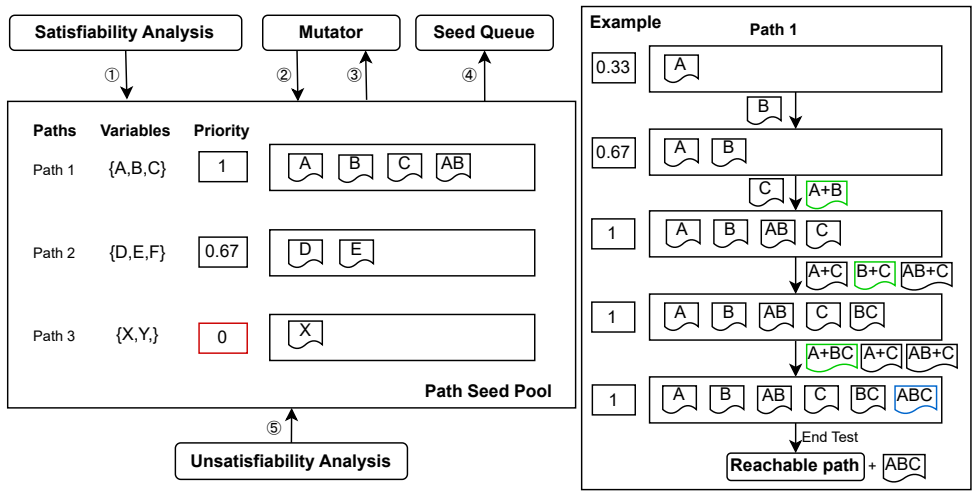


Fig. 5. Constrain aggregation. The left part shows the framework of the constraint aggregation module, and the right part presents the entire process of Path 1 from the initial state to outputting a reachable path. The green seeds represent successful aggregation, and the blue one indicates the test cases output together with the reachable path.

are considered unreachable, the vulnerability is also marked as an unreachable vulnerability, and an unreachable judgment is given.

3.5 Constraint Aggregation

In the previous section, we assessed the satisfiability of each constraint. It is important to note that the satisfiability of all constraints does not necessarily imply that the path is reachable. The satisfiable constraints must be aggregated into a single input that satisfies all constraints, in order to indicate that the path is reachable. Therefore, we propose the Path Seed Pool based on path and constraints grouping, as illustrated in the Fig.5. Each path has its own dedicated seed pool, which stores seeds that satisfy one or more constraints, along with the constraint variables and priorities for each path. As shown in step ① of Fig.5, whenever the reachability analyzer finds that a seed satisfies part of the constraints, it adds the input to the corresponding pool. For example, seeds that satisfy constraint A will be stored under the path containing A. Steps ②-④ form an aggregation process, and step ⑤ involves pruning unreachable paths.

3.5.1 Mutation of Path Seeds. The core idea behind mutating seeds during the aggregation process is that inputs satisfying disjoint constraints (e.g., A and B) can be recombined (e.g., by joining, crossing over, etc.) to generate new seeds that satisfy both constraints. As shown in the Fig.5, if seed A satisfies constraint A and seed B satisfies constraint B, they can be recombined in step ③ to form seed $A + B$. This new seed may satisfy both constraints of the target path. It is tested in the instrumented program, as shown in step ④. Eventually, we can obtain a seed that satisfies AB with high probability. This strategy allows gradual aggregation from individual constraints to all constraints along the path, producing a reachable path output.

3.5.2 Path Exploration Priority. The resource allocation between paths is dynamically adjusted, with higher priority given to paths that satisfy more constraints. Specifically, for a given path and n

constraints, if k constraints are satisfied, the path is assigned a priority of k/n. For instance, in Fig.5, when there are two paths, if path 1 has 3 constraints, all of which are satisfied, and path 2 has 3 constraints, with only one satisfied, their priorities would be 3/3 and 1/3, respectively. Normalizing these values would result in a time budget allocation of 3 : 1, meaning path 1 would receive 3/4 of the time allocation, and path 2 would receive 1/4. When no constraints are satisfied, a virtual priority of $1/n$ is assigned to the path. Pruning is also done using a priority; when the priority is set to 0, the path is considered unreachable and will not be processed or tested (step ⑤). In particular, if it is possible to satisfy all constraints of the target path, such as seed $A + BC$, the highest priority is given to that seed.

This aggregated approach allows *ConVReach* to advance each potential vulnerability path in a direction that satisfies all constraints or is unreachable, ensuring that all possible paths can be verified and that time is limited within a controlled range.

4 Implementation

ConVReach is currently implemented for C/C++ programs by operating on LLVM IR and native binaries. We implement our system as a hybrid analysis and testing framework, consisting of a static analysis part based on pysvf [SVF 2025], and a dynamic fuzzing engine built on a customized version of AFL++[AFL 2024], the system is divided into two cooperating components: the static part performs backward data-flow analysis and constraint extraction, while the dynamic part executes constraint-guided fuzzing with support for unreachable path filtering.

The Constraint Decomposition module is implemented using pysvf, a powerful static analysis framework. Pysvf provides essential utilities such as Call Graph and ICFG construction from LLVM IR, which form the backbone of our analysis. And, pysvf provides a rich set of APIs that greatly facilitate backward data-flow traversal and constraint collection at the instruction and variable level. The extractor walks backward along the ICFG to gather path constraints and data-flow constraints, which capture variable assignments and interprocedural dependencies leading to the vulnerability trigger. All collected constraints are transformed into a structured logical representation and embedded into the target binary through instrumentation.

The Constraint Satisfiability Analysis module is implemented based on constraint-guided fuzzing. The fuzzing engine is based on the improved version of the AFL++ framework. By adding a runtime-constraint interaction module to AFL++, the instrumented program can report constraint evaluation results during execution. This module interfaces with the previously inserted instrumentation logic to capture at runtime whether a single constraint, given the current input, is satisfied. In the part of unreachability analysis, we refer to the idea of Pythia [Böhme 2018] and embed a lightweight unreachability evaluation in the fuzzer, tracking the paths whose constraints are unsatisfiable for a long time. The Constraint Aggregation module maintains a list of multiple sets as a path seed pool, and each set records the information and storage path of the path seeds. The aggregation algorithm of the seeds reuses the fuzzing test mutator from the Constraint Satisfiability Analysis module.

5 Experimental Evaluation

To comprehensively evaluate *ConVReach*, we conduct the experiments focusing on both real-world and synthetic dependency-vulnerability reachability scenarios. The experiments aim to assess the effectiveness and efficiency compared with state-of-the-art baselines, and investigate the actual efficacy of internal methods (i.e., Abalation). Specifically, this section first introduces the benchmarking dataset construction, baseline selections and runtime environment. The results of experiments are organized around three key research questions:

- RQ1: How effective is *ConVReach* in real-world dependency vulnerability reachability analysis tasks?
- RQ2: How does *ConVReach* compare with the baselines in terms of performance overhead?
- RQ3: How critical are the constraint satisfiability analysis and aggregation modules of *ConVReach*? (Ablation)

5.1 Experimental Setup

5.1.1 Dataset. In this study, a special experimental scenario is designed to verify the reachability decision problem with vulnerabilities in dependencies. Due to the absence of a publicly available standard benchmark tailored to this specific application scenario, we deemed it necessary to construct a representative dataset. This dataset not only fills a gap in existing benchmarks but also facilitates a more comprehensive evaluation of reachability in the context of vulnerabilities within dependency libraries.

While the libraries included in our benchmark are selected from existing DGF benchmarks, it is crucial to note that our evaluation scenario differs significantly from conventional approaches. Existing benchmarks typically focus on assessing whether vulnerabilities can be triggered, emphasizing reachable cases. In contrast, our work evaluates whether a given application can reach vulnerable code within its dependent libraries, encompassing both reachable and unreachable cases. This distinction underscores the need for a dataset that can assess both types of scenarios, enabling a broader scope of reachability analysis.

The libraries used in our benchmark are mainly derived from common research on vulnerability analysis and dependency analysis, and include the vulnerabilities with dependency relationships identified in the compared papers. Open-source applications were selected from the projects that rely on these libraries, with the added criterion that each application has more than 10 GitHub stars. For each application and corresponding library, we used stable versions on which the application depends on the target version of the library.

The dataset includes 11 open-source applications and 7 dependency libraries, covering 53 CVE vulnerabilities associated with these libraries. It also contains 84 test cases designed to evaluate the usability of the algorithms and 347 flags inserted to simulate vulnerabilities (detailed in § 5.2).

5.1.2 Baseline. DAFL[Kim et al. 2023] and SAFUZZ[Yu et al. 2024] are selected as the baselines. At the same time, the static method Fasten[fas 2024] is added for comparison. DAFL performs slicing through the Def-Use Graph of the program to greatly reduce the search space. In several public benchmarks, the path coverage and vulnerability triggering efficiency of DAFL are greatly improved compared with the traditional directed greybox fuzzing (DGF). SAFUZZ is another recently proposed directional fuzzing framework, which also significantly outperforms the classical DGF implementation.

In the datasets used in the papers for both approaches, some of the vulnerabilities in them does not lie within the libraries, which makes them unable to be directly applied to our method. In contrast, our dataset collects cases where the vulnerability lies in the library, allowing them to be transformed into cases that can be handled by our approach.

For the static baseline, we used FASTEN's OPAL-based call-graph construction with a CHA configuration [fas 2024]. We generated a whole-program call graph for the application and its dependencies and classified a vulnerability as reachable if the function containing the vulnerable statement was reachable in the merged call graph. No additional path-feasibility or runtime filtering was applied.

In view of their leading performance in efficiency and effectiveness, we choose them as baselines to systematically compare with the proposed method to evaluate the effectiveness of dependency

vulnerability reachability analysis. For all the dynamic methods, we use seeds that are randomly taken from an open-source fuzz testing seed library [Salmonx 2021] on GitHub as the same initial seed. This ensures that these seeds conform to the input format and there are no biased selections. The experiment compares the advantages and disadvantages of different methods by comparing the performance of each method in accuracy, recall rate, F1 score and time cost.

5.1.3 Running Environment. We run experiments on a workstation equipped with an Intel Core i9-13900K CPU at 3.20GHz and 32GB of RAM running Ubuntu Linux 20.04.1 LTS.

5.2 RQ1: How Effective Is *ConVReach* in Real-World Dependency Vulnerability Reachability Analysis Tasks?

5.2.1 Setup. This experiment aims to evaluate the effectiveness of *ConVReach* in real-world dependency vulnerability reachability analysis for C/C++ programs. As no existing benchmark is available for this task, we construct a new dataset based on 11 open-source applications and 7 widely used dependency libraries. Because libraries may be used by multiple applications and applications may depend on multiple libraries, the relationship is not one-to-one. Therefore, the 11 applications and 7 libraries are organized into 15 testing pairs, where each pair represents a unique combination of an application and one of its dependency libraries.

For each testing pair, we derive testing cases by combining the CVEs of the dependency library with the application that potentially invokes the vulnerable APIs. Following the criterion that all reproducible CVEs of the selected library versions should be included, we collected 53 CVEs in total. Since different applications use dependencies in different ways, the reachability of the same CVE may vary across pairs. Consequently, vulnerability reachability is evaluated separately for each pair, resulting in 84 testing cases, of which 61 are reachable and 23 are unreachable, manually verified by two authors with cross-validation. Each test case is executed within a 24-hour time budget, repeated five times, and the median runtime is reported. For baseline methods that cannot directly determine unreachability, unresolved cases after 24 hours are conservatively treated as unreachable.

To compensate for the limited scale and structural diversity of real-world CVE datasets, we further construct a supplementary benchmark. Inspired by the Magma benchmark [Hazimeh et al. 2020], which highlights the difficulty of analyzing vulnerabilities involving deep loops, indirect function-pointer chains, and other complex semantic patterns, we insert artificial flags to simulate such challenging scenarios. Two principles guide the injection: (1) insertion locations reflect realistic vulnerability distributions (e.g., memory accesses and system interactions), and (2) injected scenarios target semantic patterns underrepresented in CVE datasets [Hazimeh et al. 2020; Hulin et al. 2017; Zhang et al. 2022]. Flags are inserted at sites including pointer dereferences, array indexing, system function calls, and logic involving input parameters or return values. The resulting benchmark, Flagbench, contains 347 testing cases across the same 7 libraries and follows the same experimental settings as the CVE-based dataset.

5.2.2 Results on CVE Dataset. As shown in Table 1, *ConVReach* successfully detected 59 out of 61 reachable vulnerabilities and all 23 unreachable ones. By comparison, DAFL and SAFUZZ detected 54 and 55 reachable vulnerabilities, respectively, within the 24-hour time budget. Although the final totals appear comparable, the difference becomes striking at earlier time intervals. At 6 hours, *ConVReach* detected 57% more reachable vulnerabilities than DAFL and 47% more than SAFUZZ. At 12 hours, this advantage persisted, while *ConVReach* had already completed nearly all detections, whereas DAFL and SAFUZZ had covered only about 60% of the cases.

FASTEN reported 14 of the 23 unreachable cases as reachable at the call-graph level. This result is expected, as FASTEN performs structural reachability analysis and conservatively treats any

Table 1. Effectiveness comparison of baselines on real-world CVE dataset, with counts of reachable/unreachable cases (left of "/": reachable; right of "/": unreachable). Values in parentheses under the "Total" row indicate the relative improvement of *ConVReach* compared with the corresponding baseline.

Program	Denpendency	Num.CVEs	*ConVReach*			DAFL			SAFUZZ			FASTEN
			6h	12h	24h	6h	12h	24h	6h	12h	24h	\
pstoedit	libming	14/4	9/2	14/4	14/4	4/-	11/-	13/-	5/-	10/-	12/-	17/1
FPlayer	ffmpeg	5/2	3/0	5/2	5/2	2/-	3/-	4/-	3/-	4/-	4/-	7/0
fanplayer	ffmpeg	4/1	3/0	3/1	3/2	2/-	2/-	3/-	2/-	3/-	4/-	4/1
QSoas	zlib	3/1	3/0	3/1	3/1	2/-	3/-	3/-	2/-	3/-	3/-	4/0
7-zip	zlib	1/3	1/1	1/3	1/3	1/-	1/-	1/-	1/-	1/-	1/-	3/1
libarchive	zlib	4/0	2/0	3/0	3/1	3/-	3/-	3/-	2/-	2/-	2/-	4/0
iipsrv	libjpeg	4/1	2/1	3/1	3/2	1/-	3/-	4/-	2/-	4/-	4/-	5/0
iipsrv	libtiff	4/2	3/0	4/1	4/2	1/-	3/-	4/-	2/-	4/-	4/-	5/1
OpenImageIO	libpng	2/2	2/2	2/2	2/2	2/-	2/-	2/-	2/-	2/-	2/-	4/0
ExifPro	libjpeg	5/0	3/0	4/1	4/1	2/-	3/-	3/-	2/-	3/-	3/-	5/0
ExifPro	libtiff	5/1	3/0	5/1	5/1	2/-	4/-	4/-	2/-	4/-	5/-	6/0
ExifPro	zlib	2/2	2/1	2/2	2/2	1/-	2/-	2/-	1/-	1/-	2/-	3/1
ExifPro	libpng	3/1	3/0	3/0	3/1	2/-	3/-	3/-	2/-	3/-	3/-	3/1
LibreOffice	libxml2	2/2	2/1	2/2	2/2	1/-	2/-	2/-	2/-	2/-	2/-	2/2
srcML	libxml2	3/1	3/0	3/1	3/1	1/-	2/-	3/-	1/-	3/-	3/-	3/1
Total		61/23	44/8	58/22	59/22	28	47	54	30	50	55	75/9
Promotion			-	-	-	(+57%)	(+23%)	(+9%)	(+47%)	(+16%)	(+7%)	-

Table 2. Effectiveness comparison of baselines on Flagbench dataset, with counts of reachable/unreachable cases (left of "/": reachable; right of "/": unreachable). Values in parentheses under the "Total" row indicate the relative improvement of *ConVReach* compared with the corresponding baseline.

Program	Denpendency	Num.flags	*ConVReach*			DAFL			SAFUZZ			FASTEN
			6h	12h	24h	6h	12h	24h	6h	12h	24h	\
pstoedit	libming	31/21	14/6	27/22	29/23	12/-	19/-	24/-	14/-	20/-	28/-	48/4
fanplayer	ffmpeg	70/44	22/14	51/35	57/57	21/-	41/-	48/-	18/-	43/-	49/-	105/9
7-zip	zlib	20/28	16/17	20/25	20/28	14/-	18/-	19/-	13/-	16/-	19/-	42/6
ExifPro	libjpeg	28/25	19/9	25/20	28/25	14/-	21/-	22/-	14/-	20/-	24/-	47/6
iipsrv	libtiff	10/19	6/5	9/12	9/20	5/-	7/-	7/-	6/-	7/-	7/-	23/6
OpenImageIO	libpng	15/10	11/6	14/10	14/11	4/-	7/-	12/-	6/-	9/-	13/-	22/3
srcML	libxml2	14/12	10/3	12/11	12/14	5/-	9/-	12/-	8/-	11/-	12/-	22/4
Total		188/159	98/60	158/135	169/178	75	122	144	79	126	152	309/38
			-	-	-	(+32%)	(+30%)	(+14%)	(+24%)	(+25%)	(+11%)	-

call-graph path as potentially executable. These cases illustrate that structural reachability alone is insufficient to determine vulnerability exploitability, and highlight the need for additional semantic analysis to refine call-graph results. In contrast, *ConVReach* and other dynamic methods produced no false positives.

Insight 1: *An interesting observation is that early-time performance reveals more than just efficiency, it also reflects how well different tools exploit program structure under tight budgets. ConVReach 's ability to quickly resolve a majority of cases suggests that reachability analysis benefits from front-loaded constraint solving, where early pruning plays an important role.*

5.2.3 Results on Flagbench. As shown in Table 2, *ConVReach* reported 19 false negatives on reachable cases. Most of these errors stemmed from the threshold-relaxation strategy, which led to premature pruning due to incorrect satisfiability judgments. Tighter thresholds could mitigate

these errors but would require significantly more time to establish unreachability, highlighting a trade-off between completeness and efficiency. Notably, these missed cases were also problematic for baseline methods.

In the 6-hour setting, *ConVReach* 's lead over baselines was smaller than on the CVE dataset, but at 12 hours, the relative advantage remained stable rather than diminishing. This suggests that the more complex program structures in Flagbench required longer preprocessing of constraints, but once completed, *ConVReach* retained its superior efficiency. Some unreachable cases timed out in this benchmark, particularly those involving constraints that appeared satisfiable in isolation but became mutually exclusive when composed with constraints from the main program. These intentionally designed scenarios highlight the intrinsic difficulty of reachability reasoning in adversarial program structures. For the static approach, performance on this test is worse than for the CVE dataset, due to the fact that fewer cases can be simply judged from the call graph in the cases built by inserting flags.

Insight 2: *The threshold relaxation strategy is effective, but may lead to erroneous pruning and false negatives for reachable cases. Proving unreachability remains challenging because mutually exclusive constraints across the caller-callee boundary often lead to timeouts.*

Answer to RQ1: The results demonstrate that *ConVReach* is highly effective for real-world reachability analysis tasks. On the CVE dataset, it achieved near-perfect coverage of both reachable and unreachable cases while significantly outperforming state-of-the-art baselines within practical time budgets. On the more challenging Flagbench dataset, *ConVReach* maintained a consistent advantage, despite encountering a small number of false negatives and timeouts in adversarially complex scenarios, which proves the effectiveness of *ConVReach*.

5.3 RQ2: How Does *ConVReach* Compare with the Baselines in terms of Performance Overhead?

5.3.1 Setup. To systematically quantify the influence of flag position on the efficiency of reachability testing, this paper conducted a comprehensive comparison among *ConVReach*, DAFL, and SAFUZZ. The evaluation was performed within a fixed running time budget of 4 hours. Multiple flags were injected at varying depths—defined by the number of basic blocks preceding each flag—while preserving the original control flow structure of the program. This approach ensured that the injections did not alter the program's inherent logic or introduce artificial complexities. For each injected flag, the elapsed time in seconds required to successfully trigger it was precisely recorded. The tested programs included ffmpeg, libtiff, libxml2, and zlib, selected for their diverse structures and real-world relevance in multimedia processing, image handling, XML parsing, and compression tasks, respectively. This setup enables a direct assessment of the performance of each tool when dealing with paths of different depths, thereby highlighting the differences in their path exploration strategies and resource utilization.

5.3.2 Result. Fig. 6 presents the test results across four programs—(a) ffmpeg, (b) libtiff, (c) libxml2, and (d) zlib—plotting the number of basic blocks before each flag on the x-axis against the time taken to detect it on the y-axis. In each subplot, the blue line represents *ConVReach*, the red line represents DAFL, and the green line represents SAFUZZ. The curves reveal distinct performance characteristics. *ConVReach*'s curve maintains a relatively flat trajectory across all tested programs,

even as flag positions extend deeper into the control flow. This flatness indicates that *ConVReach* effectively manages the dynamic search space through its path-seed pool and unreachable-path pruning mechanisms, preventing exponential growth in exploration effort and ensuring stable detection times regardless of depth.

In contrast, DAFL and SAFUZZ exhibit sharply rising curves, characterized by steep increases in time cost as the number of basic blocks grows. These steep ascents signify that both tools encounter significant efficiency bottlenecks when dealing with deeper paths: the computational overhead escalates rapidly due to unconstrained path explosion, leading to prolonged exploration times. For instance, in ffmpeg (a), DAFL's time cost surges beyond 10,000 seconds for flags past 300 blocks, approaching the 4-hour limit, while SAFUZZ plateaus at lower depths but still rises steadily. Similar patterns appear in libtiff (b), where both DAFL and SAFUZZ show exponential-like growth, with DAFL exceeding 2,000 seconds around 150 blocks. In libxml2 (c) and zlib (d), the steep rises for DAFL and SAFUZZ culminate in frequent timeouts before discovering flags beyond a few thousand blocks, underscoring their limitations in scaling to complex, deep control flows. Overall, these results demonstrate *ConVReach*'s superior scalability and efficiency in reachability testing for deep program paths.

Answer to RQ2: This evaluation demonstrates an increasingly pronounced benefit of *ConVReach* for deeper vulnerabilities. As flag depth grows, *ConVReach* 's combination of constraint-guided seed scheduling and infeasible-path elimination delivers greater reductions in time cost compared to purely dynamic directed fuzzers.

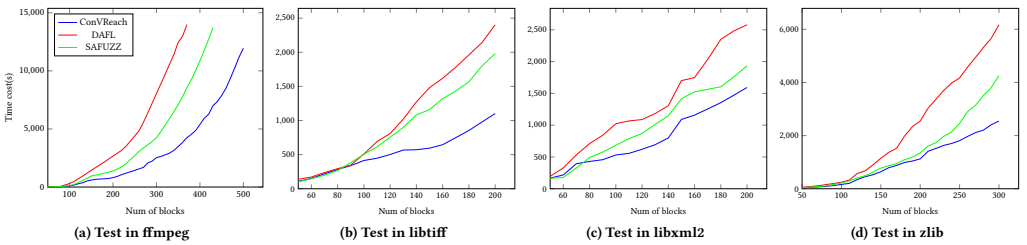


Fig. 6. Test results of time cost increasing with number of basic blocks

5.4 RQ3: How Critical Are the Constraint Satisfiability Analysis and Aggregation Modules of *ConVReach*?

5.4.1 Setup. In order to evaluate the impact of the two key modules of path seed pooling and unreachability analysis on the system performance, ablation tests are performed on the CVE dataset of real programs. For each case, we run 3 experiments separately, each with a time limit of 12 hours, and the program is considered reachable only if the vulnerability is successfully triggered in at least 2 experiments. The other experimental Settings are the same as § 5.2.

5.4.2 Result. In the removal path seed pool setting, all seeds that satisfy the variable constraints are mutated like other seeds and directly put into the seed queue without additional priority screening, grouping storage, or intra-group crossover mutation. The results show that after removing the path seed pool, only 44 cases across the three repeated experiments successfully triggered the vulnerability, and all other cases were timeouts. The false-negative rate also increased to 27.9%, and

Table 3. Ablation results. Each test contains 61 reachable cases and 23 unreachable cases from the real world.

Model Variant	R	UR	TO	False negative rate	Accuracy
ConVReach	58	26	0	0.049	0.976
w/o Path Seed Pool	44	23	17	0.279	0.798
w/o Unreachability Analyzer	35	0	49	0.426	0.690
w/o Path Seed Pool & Unreachability Analysis Module	22	0	62	0.639	0.536

there were 17 cases of timeout. It can be seen that the path seed pool effectively improves the test focusing ability and vulnerability arrival rate through grouping management, crossover mutation and priority execution of seeds that meet the variable constraints.

In the ablation experiment of removing the unreachable analyzer, the unreachable analysis phase is completely omitted, the pruning strategy related to the unreachable analysis does not work, and the fuzzing will continue to execute until the vulnerability is triggered or the test resources are exhausted. Under this setting, only 35 cases across the three experiments successfully found the corresponding vulnerability, and the false-negative rate increased to 42.6%. Because the unreachable judgment could not be given, the remaining cases all expired. It can be seen that the unreachability analyzer plays a key role in pruning invalid paths and avoiding resource waste, which significantly reduces the test search space and improves the reachability efficiency.

After removing both parts, the number of cases that could find vulnerabilities further dropped to 22 and the remaining cases all expired, reaching more than half of the false negative rate. It can be seen that the core of the dynamic part is these two components, which jointly contribute to the effect of *ConVReach*.

Answer to RQ3: From the experimental results, the path seed pool accelerates vulnerability triggering by optimizing the management and mutation of key seeds. The unreachability analyzer reduces invalid tests by excluding infeasible variable constraints and paths. The system-wide *ConVReach* achieves the lowest average test cost while maintaining the highest vulnerability trigger rate, which verifies the effectiveness of the design.

5.5 Case Study

We selected seven cases to evaluate the reachability analysis capability of *ConVReach*. Among them, *ConVReach* and the other two methods successfully verified the reachability of the cases twice, which we call A_1 and A_2; *ConVReach* successfully analyzed the reachability while the other methods timed out, with two cases being B_1 and B_2; All methods timed out in two cases, which we call C_1 and C_2; Finally, there was one and only one case where *ConVReach* and SAFUZZ failed, but only DAFL succeeded, which we call D.

Detailed timing is shown in the table 4, we summarize qualitative behaviors in representative cases. Flag-x indicates that the case comes from Flagbench, and Num.constrains represents the number of constraints in the reachable paths or potentially reachable paths found by *ConVReach* (such as C_1 and D). *ConVReach*.constrain represents the time taken by *ConVReach* to find a path where all constraints can be satisfied (but not simultaneously), with the unit being seconds, which indicates that this path will not be pruned and subsequent processing mainly involves aggregation. The number of constraints comes from the reachable paths obtained by *ConVReach* after using

Table 4. The information of the cases selected for the case study. The data marked with * indicates that it does not belong to the same path in the target program as the other data in that row.

Source of Case	Name	Num.constrains	*ConVReach*.constrain	*ConVReach*.path	DAFL	SAFUZZ
CVE-2018-5360	A_1	29	3,504	6,315	17,242	10,658
Flag-7	A_2	62	11,692*	33,484	49,423*	41,254
CVE-2018-7557	B_1	112	28,472	59,305	-	-
Flag-108	B_2	107	20,754	70,221	-	-
Flag-43	C_1	87	21,535	-	-	-
Flag-12	C_2	-	-	-	-	-
Flag-214	D	54	9,202	-	51,875*	-

other initial inputs. The last three columns represent the time taken by each method to obtain the reachable results, with the unit being seconds.

5.5.1 Analysis of the Advantages of ConVReach. In cases A1 and A2, ConVReach rapidly satisfied most path constraints and concentrated execution effort on a small number of difficult remaining constraints. This demonstrates the effectiveness of constraint decomposition and prioritized aggregation. Although alternative paths with partially satisfiable constraints were discovered, resource allocation favored paths with higher constraint coverage, enabling efficient convergence to reachable executions.

5.5.2 Cases with Complex Constraints. Cases B_1 and B_2 represent more complex scenarios compared to A_1 and A_2, with both involving over 100 constraints—specifically, 112 for B_1 originating from CVE-2018-7557 and 107 for B_2 from Flag-108, highlighting the challenges in reachability analysis for paths with high constraint volumes. In these cases, *ConVReach* successfully verifies reachability within the allocated time limits, confirming constraint satisfiability for B_1 in 28,472 seconds and completing full path aggregation in 59,305 seconds, while for B_2, it achieves satisfiability confirmation in 20,754 seconds and path verification in 70,221 seconds. In contrast, both DAFL and SAFUZZ timeout after the 24-hour (86,400 seconds) restriction, failing to produce a complete reachable path despite generating partial seeds. Analysis of the seeds produced by DAFL and SAFUZZ reveals their progress: DAFL's closest seeds satisfy 93 out of 112 constraints in B_1 and 88 out of 107 in B_2, whereas SAFUZZ's nearest completions cover 87 in B_1 and 98 in B_2, indicating that both methods make substantial headway but stall on the final, most resource-intensive constraints. For *ConVReach* specifically, its performance in these benchmark cases from Flagbench demonstrates superior robustness in handling medium-to-large constraint sets, as it finds multiple potential paths and focuses resources on key constraints, avoiding exhaustive exploration of non-viable branches. For instance, in B_1, *ConVReach* is able to determine the satisfiability of all constraints along the path before half of the total time has elapsed, thus enabling the allocation of resources to focus on the analysis of the critical path and achieving success more quickly.

5.5.3 Cases Where ConVReach Failed. Cases C_1, C_2 and D come from flag-43, flag-12, and flag-214, respectively, and have one thing in common that *ConVReach* has an error during analysis, or has timed out. Detailed examination reveals that *ConVReach* commits the same error across all three cases: inaccuracies in its unreachability analysis module. As described in §3.4.3, the unreachability analysis relies on statistical methods to compute the probability of constraint satisfiability, deeming a path probabilistically unreachable if the calculated probability falls below a 1% threshold, and the threshold will be raised to 5% later. This mechanism, while intended to introduce a judgment of unreachability through statistical methods, also creates opportunities for misjudgment, where actually satisfiable constraints are misclassified as unsatisfiable due to statistical estimates, causing

reachable paths to be pruned and leading to wrong reachability judgments or timeouts. C_2 represents an incorrect judgment caused by the need for a complex variable calculation to be equal to a specific value, and this error indicates the risk of relying on probability pruning in a complex scenario like Flagbench. Difficult constraint conditions may affect the probability calculation. To solve this problem, it is possible to increase the classification of constraint conditions, use different thresholds for different types of constraints, and ensure that *ConVReach* does not prune prematurely. For C_1 and D, although *ConVReach* successfully identified a path where all constraints were satisfiable, a timeout still occurred. This was mainly due to the same constraint conflict problem as in A_2. In these cases, *ConVReach* had a highly prioritized path for analysis, so more resources were allocated to the constraint aggregation of this path. This led to prematurely eliminating truly reachable paths. In D, *ConVReach* and SAFUZZ timed out, while DAFL successfully discovered the path where *ConVReach* prematurely pruned due to the combination of constraint satisfiability conflicts within 51875 seconds.

5.5.4 Summary and Conclusion. Overall, the problems that occurred in C_1, C_2, and D emphasize the need for a more reasonable configuration of the unreachability module, which requires combining statistical efficiency with deterministic guarantees to balance the speed and accuracy in reachability verification.

6 Discussion

6.1 Threats to Validity

The threat to validity is that our method partially uses the idea of fuzz testing, so the content of the initial seed has a significant influence on the experimental results, and the experimental results have some randomness. We adopted the same corpus independent of the vulnerability characteristics as the initial input for each method, which ensures that each method is fair at the starting stage and also guarantees that it can be handled normally when a vulnerability missing PoC is encountered. To deal with the problem of randomness, we repeat the experiment five times for each test case to reduce its impact. To reduce the influence of outliers, we use the median rather than the average of five experiments as the result of each test case. This actually requires each test case to have at least 3 valid results for the final result to be valid, and it is more convincing to provide results that can stably provide valid results rather than relying on chance.

6.2 Limitations

ConVReach adopts a staged design that combines static path-sensitive constraint decomposition with dynamic constraint validation. When static analysis encounters structures that cannot be precisely resolved, the corresponding constraints are deferred to the dynamic phase rather than conservatively approximated. While this preserves modeling precision, overall completeness depends on the effectiveness of subsequent dynamic exploration.

In corner cases involving highly sparse satisfying inputs or complex cross-boundary constraint interactions, dynamic exploration may require extended time, and statistical pruning may prematurely eliminate theoretically satisfiable paths. Thus, *ConVReach* prioritizes practical scalability and high-confidence reasoning under bounded resources over formal completeness guarantees.

6.3 Difference between *ConVReach* and Fuzzing

Our approach draws inspiration from fuzzing, which tests programs by randomly generating inputs. However, directed gray-box fuzzing focuses on mutating inputs to continuously alter program states in order to trigger crashes and uncover vulnerabilities, whereas *ConVReach* is concerned with satisfying the sequence of variable constraints associated with a vulnerability. The main difference

is how vulnerabilities are explored: fuzzing incrementally improves coverage by discovering new execution paths. Even if directed gray-box fuzzing is aware of the vulnerable location from the start, it still relies on continuous exploration within an effectively infinite input space [Böhme et al. 2017], meaning it can theoretically run indefinitely without guaranteed results. In contrast, *ConVReach* evaluates the satisfiability of vulnerability-related constraints within a bounded number of executions, identifying one or more promising candidate paths. By piecing these constraints together like a puzzle, *ConVReach* can ultimately derive a feasible reachable path or prove that all paths are unreachable. Since this process is bounded, *ConVReach* can terminate within a finite time in the vast majority of cases.

6.4 Unreachability Confidence

ConVReach introduces a statistical criterion for declaring a path unreachable, which, while not a formal proof, provides high confidence in practice. By tuning the two threshold parameters (See § 3.4.3) that govern novelty and coverage, the analysis ensures that an "unreachable" verdict is made only when the explored input space is nearly saturated. In our design, we require the discovery of new unique values (U) for a path's constraint to drop below 1%, while the estimated coverage of its value space (G) exceeds 95%. This conservative setting yields a confidence level above 99% that no satisfying inputs remain undiscovered. In other words, although we cannot guarantee unreachability in the absolute theoretical sense, the likelihood of a false "unreachable" classification is negligibly low. Moreover, with a larger time budget, these thresholds can be tightened even further to push the confidence asymptotically closer to certainty with the cost of additional analysis time.

6.5 Case of Transitive Dependency

ConVReach can naturally extend to handle transitive dependency scenarios. Theoretically, a multi-hop call sequence through intermediate libraries can be assimilated into the existing two-part path model (P and L) by treating the intermediate segment as part of either the prefix or the library sub-path. In essence, the presence of an extra layer in the call chain does not violate the constraint decomposition model. *ConVReach* 's method of extracting and solving constraints remains directly applicable if we include the intermediate calls within the vulnerable path under analysis. In implementation, this generalization can be achieved by adjusting the entry point or call chain considered for analysis. For example, if an application calls library A, and library A in turn calls the vulnerable library B, we can consider the combination of A and B as a "dependency", or we can simply regard calling library B as a "dependency" while combining library A and the application. In both cases, *ConVReach* can seamlessly traverse the extended cross-component call graph. This compatibility means that even if there are vulnerabilities in transitive dependencies, the current framework can be applied with minimal modifications - by using the static call graph data to capture the complete transitive call chain and then conducting the constraint-based reachability analysis on this chain as we usually do.

7 Related Work

Software Composition Analysis. A closely related area to reachability analysis of vulnerability is software composition analysis (SCA). Plate et al. [Plate et al. 2015] proposed a dynamic approach that determines whether vulnerable methods in dependency libraries can be reached, implemented through dynamic and static instrumentation of unit and integration tests. Building on this, Ponta et al. [Ponta et al. 2018, 2020] introduced Eclipse Steady, which identifies the reachability of vulnerable methods or code. Their approach first applies dynamic analysis to evaluate the execution of vulnerable constructs, then uses the set of executed constructs as the basis for static analysis,

thereby combining both techniques to capture all constructs relevant for vulnerability analysis. While effective, this method depends on unit or integration tests, and thus its scalability and accuracy are limited by the availability and quality of test cases. Wu et al. [Wu et al. 2023] conducted an empirical study on the impact of vulnerabilities in upstream libraries on downstream projects. They treated all modified functions in a vulnerability patch as vulnerable functions and examined whether call paths exist from downstream projects to those functions by constructing call graphs between the upstream library and downstream project. Both commercial SCA services (e.g., Snyk [Sny 2025]) and open-source tools (e.g., GitHub Dependabot [Dep 2025]) primarily identify vulnerable dependencies using NVD data. Some commercial tools, such as SourceClear [Sou 2023], additionally support reachability analysis, but comparable open-source alternatives are not available. Although these tools also analyze the reachability of vulnerabilities, most of them do not consider the runtime state of the program, so the false negative rate and false positive rate are high. *ConVReach* focuses on the runtime state of the program and eliminates infeasible paths in time, which greatly reduces the false negative rate and false positive rate.

Directed Grey-box Fuzzing. Another closely related line of research is Directed Grey-box Fuzzing (DGF). Traditional fuzzing often faces great challenges in generating effective test cases for specific vulnerabilities. To this end, AFLGo [Böhme et al. 2017] improves the efficiency of targeted testing by preferentially exploring paths closer to the target node in the Control Flow Graph (CFG) during the testing process. On this basis, Hawkeye [Chen et al. 2018] and WindRanger [Du et al. 2022] further improved the distance measurement mechanism of AFLGo, and introduced more accurate information feedback to evaluate the path distance with the target position more accurately. FuzzGuard [Zong et al. 2020] combines the deep learning model to filter out test cases that are not related to the target in the input stage, so as to effectively reduce invalid execution paths. Beacon [Huang et al. 2022] adopted program instrumentation and runtime optimization to actively prune irrelevant paths and further reduce the search space. On the other hand, DAFL [Kim et al. 2023] improves the distance-based measure method in DGF, and alleviates the problem caused by the insufficient accuracy of distance calculation. SAFUZZ [Yu et al. 2024], on the other hand, treats target priorities as dynamic changes to avoid falling into local optima during testing. These methods have made significant progress in vulnerability reachability analysis, and can gradually improve the detection accuracy of reachable vulnerabilities. However, they all rely on triggering vulnerabilities to prove the reachability of vulnerabilities, which makes it difficult to distinguish "truly unreachable vulnerabilities" from "reachable vulnerabilities that have not been triggered" [Synopsys 2022]. In contrast, *ConVReach* introduces an analysis method based on path constraint decomposition. The key idea is to decompose a complete path into several constraints and verify the satisfiability of these constraints one by one. For the unsatisfiable constraints, *ConVReach* can determine that the corresponding path is unreachable, so as to provide a high-confidence vulnerability unreachability decision.

Dependency and Polyglot Code Analysis. Recent work has explored the risks introduced by third-party code from different perspectives. Drosos *et al.* [Drosos et al. 2024] conduct a fine-grained inter-project dependency analysis of the Python ecosystem, showing that a large fraction of dependency code is never exercised at the method level, yet still contributes to code bloat and expands the attack surface. Their work focuses on cross-project reachability at the scripting-language level, without modeling the low-level semantics of native code or the constraints governing unsafe operations. Another work, CHARON [Scholtes et al. 2025], targets vulnerabilities in native extensions of scripting languages by performing inter-procedural, cross-language static analysis. By unifying script and C/C++ code into a polyglot property graph, CHARON can reason about how script-level input validation mitigates native vulnerabilities, but relies primarily on pattern-based checks and conservative data-flow reasoning rather than explicit constraint solving.

In contrast to these approaches, *ConVReach* deliberately focuses on C/C++ code and its explicit low-level semantics, which enables the application of more precise constraint-based reachability analysis. By modeling arithmetic relations, pointer semantics, and path conditions directly in C/C++, *ConVReach* can propagate and solve constraints along execution paths to determine whether dangerous operations are truly reachable. This focus allows *ConVReach* to support constraint reasoning that is difficult to achieve in cross-language or high-level dependency analyses, positioning it as complementary to prior work when precise path feasibility and constraint expressiveness are required.

8 Conclusion and Future Work

In this paper, we propose and implement the *ConVReach* tool for analyzing the reachability of vulnerabilities in dependencies and conduct extensive experimental evaluations on CVE datasets of real programs and Flagbench datasets constructed from inserted flags. Experimental results show that compared with the existing dynamic vulnerability reachability analysis methods, *ConVReach* has higher accuracy and better performance efficiency. Through further experiments in the case of overhead, it is found that the advantages of this method are more obvious in the case of more vulnerability path constraints.

Future work can explore more directions to enhance scalability and functionality. A promising direction is to achieve concurrent analysis of multiple vulnerabilities by integrating the same constraints of multiple vulnerabilities to simultaneously verify their reachability. This has the potential to significantly shorten the analysis time for complex projects with multiple similar vulnerabilities. Another direction involves adding modular extensions to expand functionality, such as integrating a vulnerability propagation module to track the chain effects generated through transitive dependency relationships, or embedding an automatic repair hook to suggest patches or isolation strategies when an reachable vulnerability is detected. Additionally, developing plugins for seamless integration with SCA tools can facilitate real-time monitoring in the CI/CD pipeline and reduce false positives in the production environment. These improvements will make *ConVReach* a more versatile framework, capable of addressing threats such as supply chain attacks and capable of handling the increasing complexity of modern software ecosystems.

Another promising direction is to integrate *ConVReach* with symbolic execution techniques such as SMT solving, so as to leverage the complementary strengths of both approaches. SMT solvers can be used as a lightweight pre-processing step to quickly eliminate trivially infeasible paths or confirm feasible ones, thereby reducing the search space before the dynamic analysis. For the SMT solver itself, if the constraints it encounters are too complex or cause it to time out, the constraint satisfiability analysis of *ConVReach* can be integrated into the SMT solver as a backup solution, providing a time-consuming but effective feasibility judgment.

9 Data-Availability Statement

The test versions of the datasets and code used in this study are available and can be accessed at https://github.com/ianorz/ConVReach. This version allow the reproduction of the results through executable binary files. The dataset and source code are functional and have been prepared for public release through a website. Additionally, the comparison method DAFL artifacts can be viewed at https://github.com/prosyslab/DAFL-artifact, and SAFUZZ can be viewed at https://github.com/SaFuzztool/SAFuzz. All these resources will be formally submitted during the Artifact Evaluation process.

Acknowledgments

This work was supported by the National Natural Science Foundation of China under Grant No. 62332005 (Key Program Project: Trustworthiness Analysis and Assurance Methods for Open Source Software), partially supported by a Discovery Early Career Researcher Award Fellowship (DE250100192) awarded by the Australian Research Council (ARC) to Yuekang Li, the National Science Foundation of China under Grants (U22B2027), the National Natural Science Foundation of China (U24A6009), and supported by the Joint Research Center for System Security, Tsinghua University (Institute for Network Sciences and Cyberspace), and Science City (Guangzhou) Digital Technology Group Co., Ltd.

References

2021. Eclipse steady. https://projects.eclipse.org/projects/technology.steady.

2023. Software composition analysis for devSecOps. https://www.sourceclear.com/.

2024. AFL++. https://github.com/AFLplusplus/AFLplusplus.

2024. FASTEN project. https://github.com/fasten-project/fasten.

2025. Dependabot. https://github.com/dependabot/dependabot-core.

2025. Pysvf. https://github.com/SVF-tools/SVF-Python.

2025. Snyk. https://snyk.io/.

Mahmoud Alfadel, Diego Elias Costa, Emad Shihab, and Bram Adams. 2023. On the discoverability of npm vulnerabilities in node.js projects. *ACM Transactions on Software Engineering and Methodology* 32, 4 (2023), 1–27. doi:10.1145/3571848

Marcel Böhme. 2018. STADS: Software Testing as Species Discovery. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 27, 2 (2018). doi:10.1145/3210309

Marcel Böhme, Danushka Liyanage, and Valentin Wüstholz. 2021. Estimating residual risk in greybox fuzzing. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 230–241. doi:10.1145/3468264.3468570

Marcel Böhme, Van-Thuan Pham, Manh-Dung Nguyen, and Abhik Roychoudhury. 2017. Directed greybox fuzzing. In *Proceedings of the 2017 ACM SIGSAC conference on computer and communications security*. 2329–2344. doi:10.1145/3133956.3134020

Hongxu Chen, Yinxing Xue, Yuekang Li, Bihuan Chen, Xiaofei Xie, Xiuheng Wu, and Yang Liu. 2018. Hawkeye: Towards a desired directed grey-box fuzzer. In *Proceedings of the 2018 ACM SIGSAC conference on computer and communications security*. 2095–2108. doi:10.1145/3243734.3243849

Georgios-Petros Drosos, Thodoris Sotiropoulos, Diomidis Spinellis, and Dimitris Mitropoulos. 2024. Bloat beneath Python's Scales: A Fine-Grained Inter-Project Dependency Analysis. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 2584–2607. doi:10.1145/3660821

Zhengjie Du, Yuekang Li, Yang Liu, and Bing Mao. 2022. Windranger: A directed greybox fuzzer driven by deviation basic blocks. In *Proceedings of the 44th International Conference on Software Engineering*. 2440–2451. doi:10.1145/3510003.3510197

Soline Ducousso, Sébastien Bardin, and Marie-Laure Potet. 2023. Adversarial Reachability for Program-level Security Analysis.. In *ESOP*. 59–89. doi:10.1007/978-3-031-30044-8_3

Fluid Attacks. 2025. What is reachability analysis? https://fluidattacks.com/cybersecurity-essentials/what-is-reachability-analysis.

Darius Foo, Jason Yeo, Hao Xiao, and Asankhaya Sharma. 2019. The dynamics of software composition analysis. *arXiv preprint arXiv:1909.00973* (2019). doi:10.48550/arXiv.1909.00973

Ahmad Hazimeh, Adrian Herrera, and Mathias Payer. 2020. Magma: A ground-truth fuzzing benchmark. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* 4, 3 (2020), 1–29. doi:10.1145/3428334

Allen D Householder, Jeff Chrabaszcz, Trent Novelly, David Warren, and Jonathan M Spring. 2020. Historical analysis of exploit availability timelines. In *13th USENIX Workshop on Cyber Security Experimentation and Test (CSET 20)*.

Heqing Huang, Yiyuan Guo, Qingkai Shi, Peisen Yao, Rongxin Wu, and Charles Zhang. 2022. Beacon: Directed grey-box fuzzing with provable path pruning. In *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, 36–50. doi:10.1145/3510003.3510143

Patrick Hulin, Andy Davis, Rahul Sridhar, Andrew Fasano, Cody Gallagher, Aaron Sedlacek, Tim Leek, and Brendan Dolan-Gavitt. 2017. {AutoCTF}: Creating diverse pwnables via automated bug injection. In *11th USENIX Workshop on Offensive Technologies (WOOT 17)*.

Nasif Imtiaz, Seaver Thorn, and L. Williams. 2021. A comparative study of vulnerability reporting by software composition analysis tools. *Proceedings of the 15th ACM / IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)* (2021). doi:10.1145/3475716.3475769

Jay Jacobs, Sasha Romanosky, Benjamin Edwards, Idris Adjerid, and Michael Roytman. 2021. Exploit prediction scoring system (epss). *Digital Threats: Research and Practice* 2, 3 (2021), 1–17. doi:10.1145/3436242

Brittany Johnson, Yoonki Song, Emerson Murphy-Hill, and Robert Bowdidge. 2013. Why don't software developers use static analysis tools to find bugs?. In *2013 35th International Conference on Software Engineering (ICSE)*. 672–681. doi:10.1109/ICSE.2013.6606613

Tae Eun Kim, Jaeseung Choi, Kihong Heo, and Sang Kil Cha. 2023. {DAFL}: Directed Grey-box Fuzzing guided by Data Dependency. In *32nd USENIX Security Symposium (USENIX Security 23)*. 4931–4948.

Chengwei Liu, Sen Chen, Lingling Fan, Bihuan Chen, Yang Liu, and Xin Peng. 2022. Demystifying the vulnerability propagation and its evolution via dependency trees in the npm ecosystem. In *Proceedings of the 44th International Conference on Software Engineering*. 672–684. doi:10.1145/3510003.3510143

Ivan Pashchenko, Henrik Plate, Serena Elisa Ponta, Antonino Sabetta, and Fabio Massacci. 2018. Vulnerable open source dependencies: Counting those that matter. In *Proceedings of the 12th ACM/IEEE international symposium on empirical software engineering and measurement*. 1–10. doi:10.1145/3239235.3268920

Ivan Pashchenko, Henrik Plate, Serena Elisa Ponta, Antonino Sabetta, and Fabio Massacci. 2020. Vuln4real: A methodology for counting actually vulnerable dependencies. *IEEE Transactions on Software Engineering* 48, 5 (2020), 1592–1609. doi:10.1109/TSE.2020.3025443

Henrik Plate, Serena Elisa Ponta, and Antonino Sabetta. 2015. Impact assessment for vulnerabilities in open-source software libraries. In *2015 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 411–420. doi:10.1109/ICSM.2015.7332492

Serena Elisa Ponta, Henrik Plate, and Antonino Sabetta. 2018. Beyond metadata: Code-centric and usage-based analysis of known vulnerabilities in open-source software. In *2018 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 449–460. doi:10.1109/ICSME.2018.00054

Serena Elisa Ponta, Henrik Plate, and Antonino Sabetta. 2020. Detection, assessment and mitigation of vulnerabilities in open source dependencies. *Empirical Software Engineering* 25, 5 (2020), 3175–3215. doi:10.1007/s10664-020-09830-X

Grigore Rosu, Andrei Stefanescu, Stefan Ciobâcá, and Brandon M. Moore. 2013. One-Path Reachability Logic. In *2013 28th Annual ACM/IEEE Symposium on Logic in Computer Science*. 358–367. doi:10.1109/LICS.2013.42

Salmonx. 2021. Comprehensive croups for fuzzing seeds with carfefully selected. https://github.com/salmonx/seeds.

Raoul Scholtes, Soheil Khodayari, Cristian-Alexandru Staicu, and Giancarlo Pellegrino. 2025. CHARON: Polyglot code analysis for detecting vulnerabilities in scripting languages native extensions. In *2025 IEEE 10th European Symposium on Security and Privacy (EuroS&P)*. IEEE, 153–168. doi:10.1109/EuroSP63326.2025.00018

Qin Ying Wang Shouling Ji, Binbin Zhao Anying Chen, Xuhong Zhang Tong Ye, Jianwei Yin Jingzheng Wu, Yun Li, and Yanjun Wu. 2022. Survey on Open-source Software Supply Chain Security. *Journal of Software* 34, 3 (2022), 1330–1364.

Snyk. 2023. Software composition analysis (SCA): What is it and does your company need it? https://snyk.io/blog/what-is-software-composition-analysissca-and-does-my-company-need-it/.

Andrei Stefănescu, Daejun Park, Shijiao Yuwen, Yilong Li, and Grigore Roşu. 2016. Semantics-based program verifiers for all languages. *ACM SIGPLAN Notices* 51, 10 (2016), 74–91. doi:10.1145/3022671.2984027

Synopsys. 2022. "Open source security and risk analysis" (OSSRA) report. https://www.synopsys.com/software-integrity/resources/analyst-reports/open-source-security-risk-analysis.html.

Yulun Wu, Zeliang Yu, Ming Wen, Qiang Li, Deqing Zou, and Hai Jin. 2023. Understanding the threats of upstream vulnerabilities to downstream projects in the maven ecosystem. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 1046–1058. doi:10.1109/ICSE48619.2023.00095

Soufian El Yadmani, Robin The, and Olga Gadyatskaya. 2022. Beyond the Surface: Investigating Malicious CVE Proof of Concept Exploits on GitHub. *arXiv preprint arXiv:2210.08374* (2022). doi:10.48550/arXiv.2210.08374

Awad Younis, Yashwant K Malaiya, and Indrajit Ray. 2016. Assessing vulnerability exploitability risk using software properties. *Software Quality Journal* 24, 1 (2016), 159–202. doi:10.1007/s11219-015-9274-6

Awad A Younis, Yashwant K Malaiya, and Indrajit Ray. 2014. Using attack surface entry points and reachability analysis to assess the risk of software vulnerability exploitability. In *2014 IEEE 15th International Symposium on High-Assurance Systems Engineering*. IEEE, 1–8. doi:10.1109/HASE.2014.10

Xinglin Yu, Hongliang Liang, and Chunlin Wang. 2024. Multiple Targets Directed Greybox Fuzzing: From Reachable to Exploited. In *SANER 2024: Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (Software Security Track)*. Pisa, Italy. doi:10.1109/SANER60148.2024.00099

Ao Zhang, Yiying Zhang, Yao Xu, Cong Wang, and Siwei Li. 2024b. Machine Learning-Based Fuzz Testing Techniques: A Survey. *IEEE Access* 12 (2024), 14437–14454. doi:10.1109/ACCESS.2023.3347652

Fangyuan Zhang, Lingling Fan, Sen Chen, Miaoying Cai, Sihan Xu, and Lida Zhao. 2024a. Does the Vulnerability Threaten Our Projects? Automated Vulnerable API Detection for Third-Party Libraries. *IEEE Transactions on Software Engineering* 50, 11 (2024), 2906–2920. doi:10.1109/TSE.2024.3454960

Zenong Zhang, Zach Patterson, Michael Hicks, and Shiyi Wei. 2022. {FIXREVERTER}: A realistic bug injection methodology for benchmarking fuzz testing. In *31st USENIX Security Symposium (USENIX Security 22)*. 3699–3715.

Lida Zhao, Sen Chen, Zhengzi Xu, Chengwei Liu, Lyuye Zhang, Jiahui Wu, Jun Sun, and Yang Liu. 2023. Software composition analysis for vulnerability detection: An empirical study on Java projects. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 960–972. doi:10.1145/3611643.3616299

Peiyuan Zong, Tao Lv, Dawei Wang, Zizhuang Deng, Ruigang Liang, and Kai Chen. 2020. {FuzzGuard}: Filtering out unreachable inputs in directed grey-box fuzzing through deep learning. In *29th USENIX security symposium (USENIX security 20)*. 2255–2269.